

数据结构

您需要写一种数据结构来维护一些数，其中需要提供以下操作：

1. 插入 x 数
2. 删除 x 数(若有多个相同的数，因只删除一个)
3. 查询 x 数的排名(排名定义为比当前数小的数的个数+1。若有多个相同的数，因输出最小的排名)
4. 查询排名为 x 的数
5. 求 x 的前驱(前驱定义为小于 x ，且最大的数)
6. 求 x 的后继(后继定义为大于 x ，且最小的数)

有序表

根据上述操作，我们不难想到暴力的做法，因为我们需要知道一个数 x 在数列中的位置，已知他的前驱和后继，那么我们维护一个有序表就可以了。即数列是有序的，每次把数插入到对应的位置，我们来思考一下时间复杂度：

- (1) 插入数 x ：因为是有有序表，所以我们可以考虑二分查找，找到位置的时间复杂度为 $O(\log n)$ ，但是每次插入后面的数又要向后挪一位，所以最坏时间复杂度为 $O(n)$
- (2) 删除数 x ：和插入相同。
- (3) 数 x 的排名：二分查找， $O(\log n)$
- (4) 排名为 x 的数：下标查找 $O(1)$
- (5) x 的前驱，二分查找， $O(\log n)$
- (6) x 的后继，同前驱。

有序表

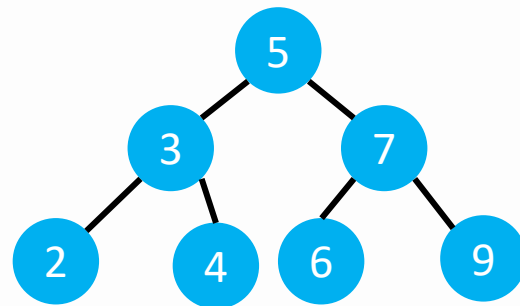
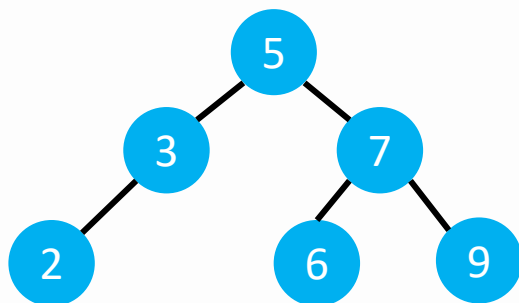
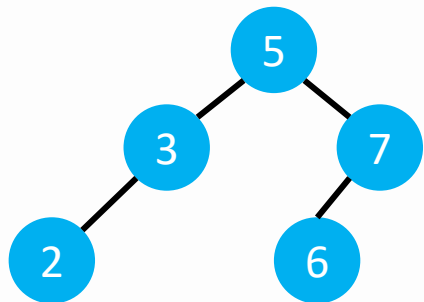
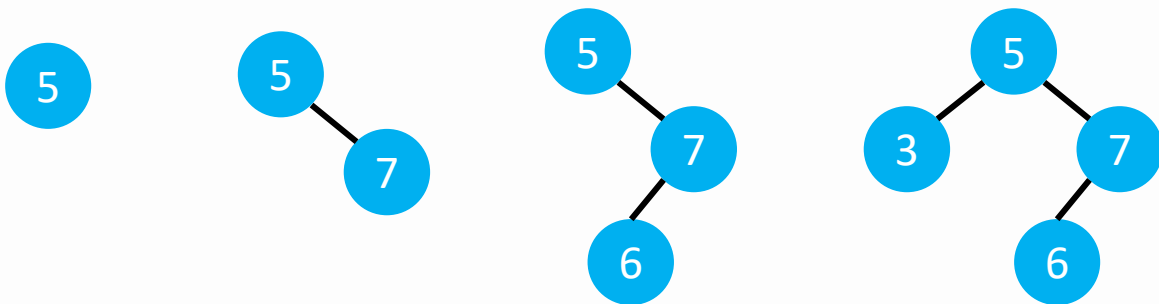
对于季赛来说， $\log n$ 的算法就已经很优秀了，但是在插入和删除上的低效率就决定了被淘汰的命运。我们再思考一下如何在插入删除上优化，对于顺序表，插入和删除都是 $O(n)$ 的，并且又要可以使用二分的思想，所以我们只能考虑树形结构。

比如堆，堆的插入删除虽然是 $\log n$ 的，但是堆没有办法直接取中间任意元素，只能访问修改队首，也不符合我们的操作。

我们可以考虑来维护一颗有序的树，即该数的左子树一定比根结点小，右子树一定比根结点数大。这样当我们中序遍历这棵树的时候就得到了一个有序表。

有序树

比如 5 7 6 3 2 9 4，建树的过程为



二叉搜索树

对于这样的一棵有序树，也叫做二叉搜索树，我们来看一下执行前面6个操作的效率：

(1) 插入：因为二叉搜索树有一个性质，即根的左子树的权值都小于根，根的右子树的权值都大于根，所以我们每次插入可以根据值的大小和当前根的大小依次到左右子树去查找，然后每次添加到树的空位置。所以时间复杂度与树的深度有关，接近于 $\log n$ 。

(2) 删除：删除比较复杂，先不讲

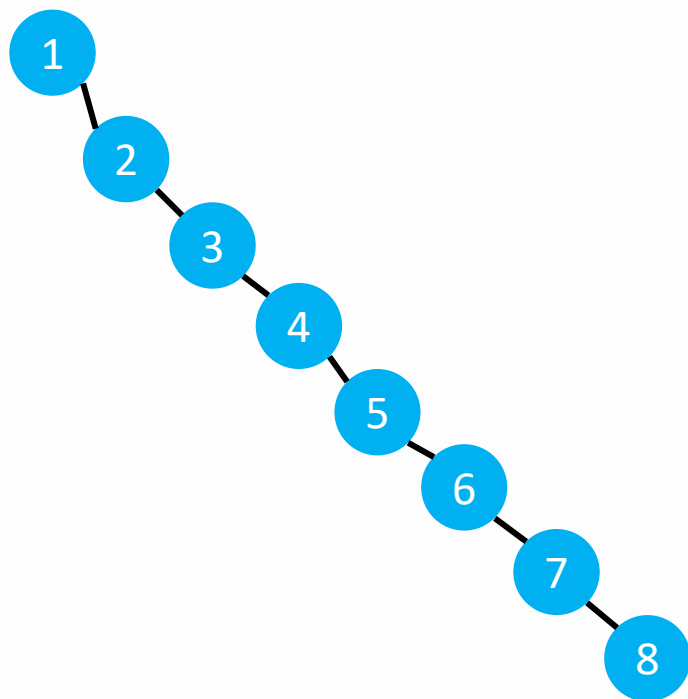
(3) 查找数 x 和排名为 x 的数，对于每一个结点我们记录当前这棵子树的个数，我们可以很快的查询到，时间复杂度接近 $\log n$

(4) 前驱与后继：差不多的原理

二叉搜索树

貌似使用这样一颗二叉搜索树来维护这六个操作比较完美，但是我们来看下面一组数据：

1 2 3 4 5 6 7 8.



看出什么问题来了吗？如果给定的序列是有序的或者大部分有序的，那么这棵树的深度就会特别深，甚至退化成一条链，当一颗树越接近完全二叉树的时候，效率会达到最高， $\log n$ ，但是如果退化成一条链的时候，效率就会讲到最低， n 。

二叉搜索树

所以我们要想办法让这棵二叉搜索树的深度不会太大，也就是更接近于一颗完全二叉树，这样效率才会更高。也就是说我们需要来维护这棵二叉搜索树的平衡，所以这类数据结构又叫做平衡树。

平衡树和我们前面学习的线段树不太一样，他是一类数据结构，而不是一种，因为有很多dalao都想出了自己的维护二叉搜索树平衡的办法，所以维护平衡的办法有很多种，每一种都有属于他们自己的名字。不过贪多嚼不烂，所以我们只需要学习其中一种或者几种就可以了。

(伪) 平衡树

为什么叫伪平衡树呢？因为实际上他本来就不是一棵二叉搜索树，不过由于的特殊性，可以让他的效率远高于有序表，但是却低于 $\log n$ 。关键是他代码特别短，很容易实现，所以是一个拿来骗分的神技——Vector。

Vector

头文件 `#include<vector>`

向量的定义：

`vector<int> vec;` //定义一个vec型的向量a

`vector<int> vec(5);` //定义一个初始大小为5的向量

`vector<int> vec(5, 1);` //初始大小为5，值都为1的向量

因为vector是可以根据插入的值的数量自动增长的，所以初始定义的时候只需要按照**第一种**方法定义就可以了（一维）。

很多时候我们需要开一个二维数组：

`vector<vector<int>> vec(100);`

Vector 操作

Vector的下标和数组一样从0开始的

`vec.size()`; //返回向量的实际大小

`vec.begin()`; //返回向量的开始指针的位置

`vec.end()`; //返回向量的结束指针的下一个位置

`vec.push_back(x)`; //在对象末尾插入数据x

`vec.pop_back()`; //在对象末尾删除数据

`vec.clear()`; //清除对象中的所有数据

`vec.at(i)`; //访问容器中第i个数的值(推荐)

`vec[i]`; //访问容器中第i个数的值(不推荐)

Vector

```
#include<vector>
using namespace std;
vector<int> a;
int main()
{
    int n,m,tmp;
    scanf("%d%d",&n,&m);
    for(int i=1;i<=n;i++)
    {
        scanf("%d",&tmp);
        a.push_back(tmp); //插入操作
    }
    sort(a.begin(),a.end()); //排序, 不能用a+1,a+n+1
    for(int i=1;i<=m;i++)
    a.pop_back(); //删除操作
    for(int i=0;i<a.size();i++)
    printf("%d ",a.at(i)); //访问a中第i个元素
    // printf("%d ",a[i]);
    printf("\n");
}
```

Vector

在第 $i+1$ 个数前面插入一个数 x :

```
vec.insert(vec.begin()+i, x);
```

删除第 $i+1$ 个数:

```
vec.erase(vec.begin()+i);
```

这才是vector的精髓所在，以上删除，插入操作都是接近 $\sqrt{n}$ 的，因为vector下标是从0开始的，所以下标为 i 的数实际上就是第 $i+1$ 个数
以上两个操作再集合两个STL函数可以1行代码实现一个平衡树的单点操作。

数据结构

您需要写一种数据结构来维护一些数，其中需要提供以下操作：

1. 插入 x 数
2. 删除 x 数(若有多个相同的数，因只删除一个)
3. 查询 x 数的排名(排名定义为比当前数小的数的个数+1。若有多个相同的数，因输出最小的排名)
4. 查询排名为 x 的数
5. 求 x 的前驱(前驱定义为小于 x ，且最大的数)
6. 求 x 的后继(后继定义为大于 x ，且最小的数)

普通平衡树

操作一：插入数 x

在vector中，insert的插入时间复杂度为 $\log n$ 的（insert:在第 i 个数之前插入一个数），，所以我们可以用insert插入，在插入之前需要先找到插入的位置，即第一个比数 x 大的数之前，所以可以使用upper_bound（）；

```
a.insert(upper_bound(a.begin(), a.end(), x), x);
```

这里也可以使用lower，相同的数，位置不影响

普通平衡树

操作二：删除数x

在vector中，erase的删除时间复杂度为，在删除之前需要先找到x的位置，因为保证了数x一定存在，那么我们可以使用lower_bound()，这里不可以使用upper_bound()，因为如果用后一个函数，正确答案是查找的数的前一个数，但是vector不是连续的（类似链表），所以不能直接-1。

```
a.erase(lower_bound(a.begin(), a.end(), x));
```

普通平衡树

操作三：查询数x的排名

因为我们插入的时候都是保证了元序列的有序性，所以数x的下标+1就是数x的排名，所以我们只需要找到数x的下标就可以了，并且是第一次出现的位置。

`lower_bound(a.begin(), a.end(), x) - a.begin() + 1`

普通平衡树

操作五：求数x的前驱

我们只需要找到第一个等于x的数，然后他前面一个数一定是x的前驱。

```
tmp=lower_bound(a.begin(), a.end(), x)-a.begin()-1)
```

```
a.at(tmp)
```

普通平衡树

操作六：求数 x 的后继

我们只需要找到第一个大于 x 的数。

```
tmp=upper_bound(a.begin(), a.end(), x)-a.begin()
```

```
a.at(tmp)
```

Multiset

接下来讲一个平衡树的STL，准确的来说是红黑树的Stl。Set和Map都是基于红黑树的STL，不过两者用途不太一样。用平衡树的操作一般用Set。不过Set有一个问题，那就是键值唯一，也就是说对于有重复的值只会保留一个，所以我们要用Multiset，他是允许存在重复元素的。

但是无论是Set还是Multiset，他都是严格的平衡树，所以他是不支持直接访问第几个元素的。也就是普通平衡树这道题如果要用multiset写貌似很麻烦，好像还需要自己写一个二分。

Multiset

需要头文件: `#include<set>`

`multiset<int> t;`

`t.begin()`: 返回容器第一个迭代器

`t.end()`: 返回容器最后一个迭代器

`t.clear()`: 删除容器中所有元素

`t.empty()`: 判断容器是否为空

`t.size()`: 返回容器中当前元素的个数

`t.insert(x)`: 插入到合适的位置

`t.erase(a.begin()+i)`: 删除第 $i+1$ 个数

`t.erase(a.begin()+i, a.begin()+j)`: 区间删除 $[i, j)$

`t.erase(x)`: 删除数字 x

`t.find(x)`: 查找 x 出现的位置, 没找到返回 `s.end()`

Multiset

1: 插入:

`t.insert(x)`: 插入数 x :

2: 删除: 有三种

`t.erase(a.begin()+i)`: 删除第 $i+1$ 个数

`t.erase(a.begin()+i, a.begin()+j)`: 区间删除 $[i, j)$

`t.erase(x)`: 删除所有的数字 x

3: 定义迭代器(指针)

`multiset<int>::iterator it;`

`it=t.lower_bound(x);`

4: 查询具体值

`t1=*(t.lower_bound(x));`

Multiset

`t.lower_bound(k)`: 返回第一个大于等于k的元素的位置

`t.upper_bound(k)`: 返回第一个大于k的元素的位置

Multiset的遍历比较麻烦，记住即可：

```
multiset<int>::iterator it;//定义一个迭代器名为it  
for(it=t.begin();it!=t.end();it++)  
printf("%d",*it);
```

Multiset

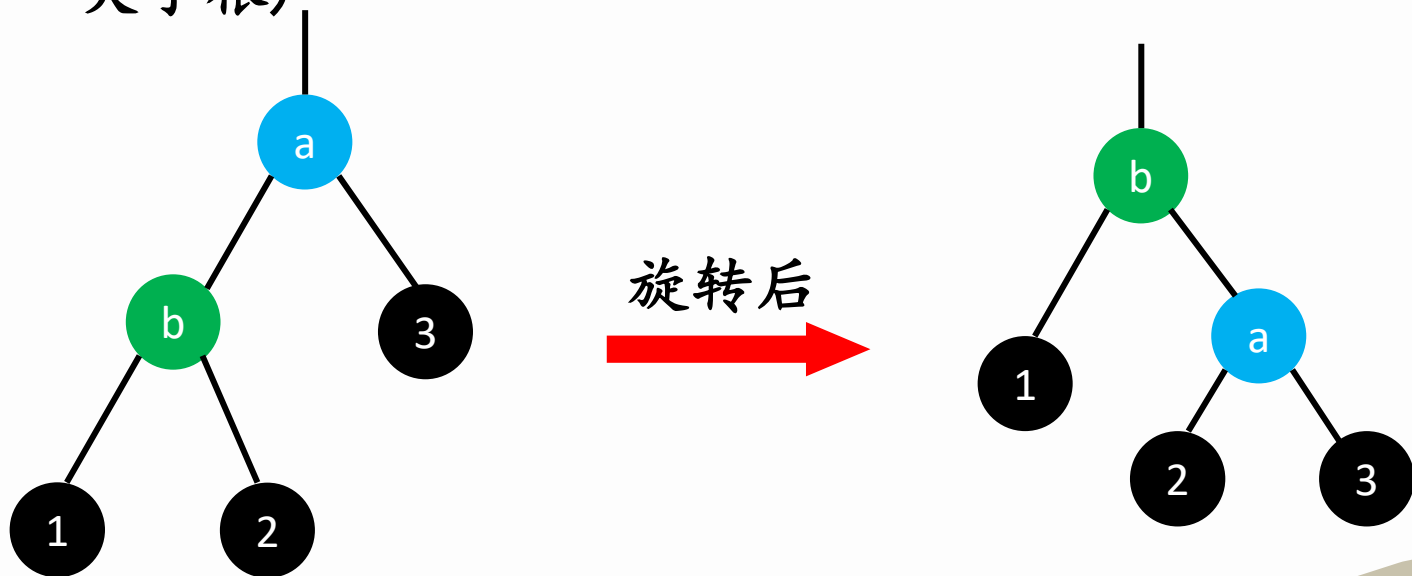
```
int main()
{
    int x,n;
    multiset<int> t;
    scanf("%d",&n);
    for(int i=1;i<=n;i++)
    {
        scanf("%d",&x);
        t.insert(x);
    }
    scanf("%d",&x);//删除数x
    t.erase(t.find(x));//找到x的位置再删除该位置
    multiset<int>::iterator it;
    for(it=t.begin();it!=t.end();it++)//遍历
    printf("%d ",*it);
    return 0;
}
```

平衡树

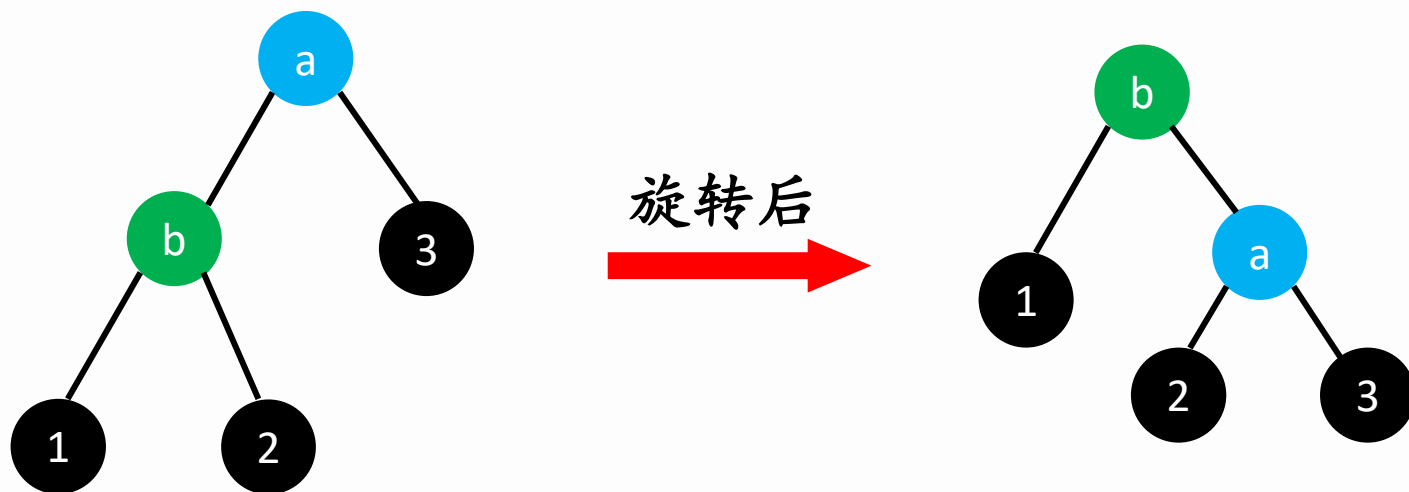
维护平衡树的平衡的方式有多重，常见的有两种，第一种是旋转，比如Treap和Splay，另一种是不旋转，比如非旋Treap。今天我们先讲解Splay，因为Splay的使用范围更广，并且支持区间操作，而Treap不支持区间操作，所以他的功能都可以通过STL或者Vector来实现。

旋转

基于旋转的平衡树，最核心的地方就是旋转。简单来说就是交换父子关系，也就是本来a是b的父亲结点，我们现在把a变成b的子节点，我们来观察一下要如何旋转才能让平衡树保持原性质不改变(左子树小于根，右子树大于根)



旋转



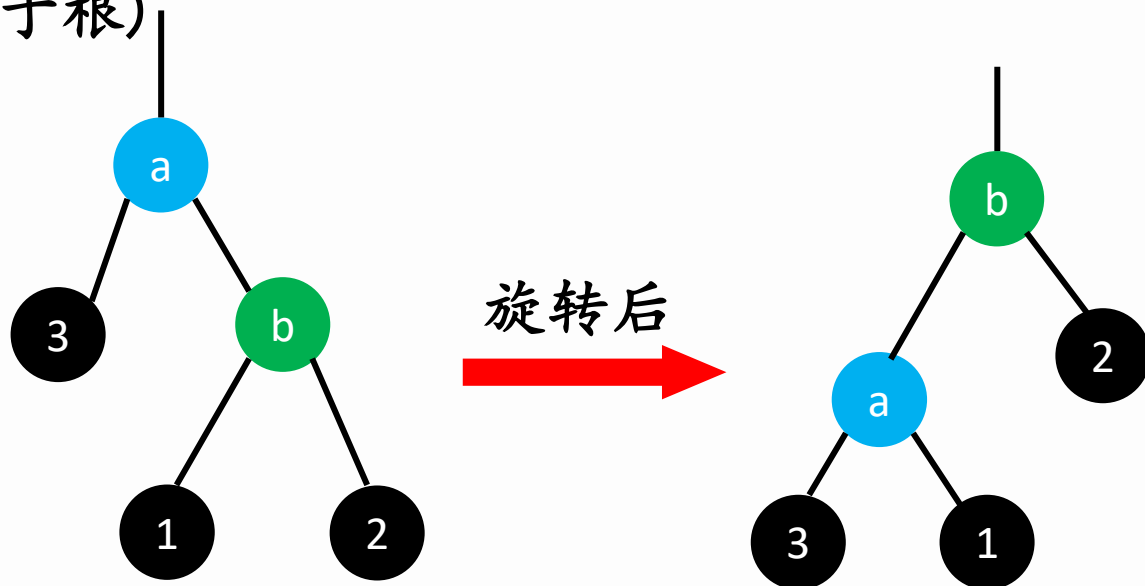
如果b是a的左儿子，我们就可以写出旋转的代码：

```
int tmp=b.rchild;  
b.rchild=a;  
a.lchild=tmp;
```

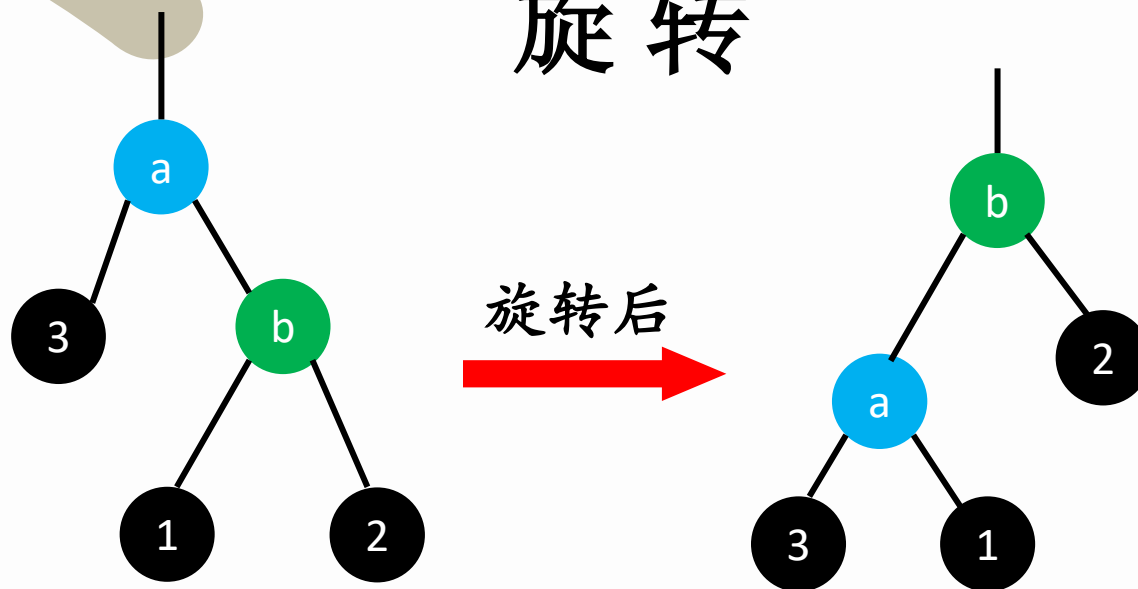
右旋

旋转

基于旋转的平衡树，最核心的地方就是旋转。简单来说就是交换父子关系，也就是本来a是b的父亲结点，我们现在把a变成b的子节点，我们来观察一下要如何旋转才能让平衡树保持原性质不改变(左子树小于根，右子树大于根)



旋转



如果b是a的右儿子，我们就可以写出旋转的代码：

```
int tmp=b.lchild;  
b.lchild=a;  
a.rchild=tmp;
```

左旋



练习题

洛谷	3369	普通平衡树(Vector, 平衡树)
洛谷	3391	文艺平衡树
洛谷	2286	宠物收养场
洛谷	4146	序列终结者
洛谷	2161	会场预约
洛谷	1110	报表统计
洛谷	3960	列队(平衡树)

